

USO DEL MODELO EN LA RASPBERRY PI PICO

Para emplear nuestro modelo en la pico y realizar la inferencia, según los autores de tinyML lo que recomiendan es, que si se trata de un proyecto para empezar y sencillo, utilizar los repositorios que ellos mismos proporcionan y simplemente modificar el código para que cumpla nuestras necesidades.

Todas las librerías y códigos que se requieren, nos las proporciona el siguiente enlace: <https://github.com/raspberrypi/pico-tflmicro/>

Inicialmente para nuestro proyecto, solo nos interesa probar que funcione la inferencia del modelo, por lo tanto con probar una imagen valdría. Posteriormente habrá que añadir al código el manejo de las imágenes proporcionadas por la cámara que conectemos a la pico.

Tomaremos como ejemplo el proyecto llamado hello_world, que se encarga de realizar una inferencia con un modelo entrenado para predecir la función del seno. Tomando dicha estructura, crearemos la estructura de nuestro proyecto “deteccionCaracol”:

- main.cpp: Llama a las funciones setup() y loop().
- main_functions.cpp: Implementa las funciones setup() y loop() (Detallado su funcionamiento en el código).
- imagen_prueba.cpp: Se guarda la imagen transformada a 96x96, RGB (3 canales) y cada pixel podrá tomar un valor entero entre 0 y 255. Esta imagen se convertirá en un array de $96 \times 96 \times 3 = 27\,648$. Los valores de cada pixel se obtienen con el código “conversion_imagenes.py”.
- modelo_data.cpp: Es el modelo de mobileNetV2 entrenado, que hemos convertido en un array de bytes en formato C.
- CmakeLists.txt: Indica qué archivos hay que compilar y enlazar a las librerías de pico-tflmicro. También genera el archivo .uf2 que hay añadir en la Raspberry Pi Pico.

Con estos ficheros estamos listos para realizar la primera inferencia, pero antes será necesario compilar y generar el ejecutable:

1º. Asegurarnos de que está guardada la variable \$PICO_SDK_PATH con “echo \$PICO_SDK_PATH”. En caso de que no esté, la estableceremos, habrá que hacer algo probablemente parecido a (Depende de donde se encuentre instalada la SDK):

```
“echo 'export PICO_SDK_PATH=/home/sandra/.pico-sdk/sdk/2.1.0' >> ~/.bashrc  
source ~/.bashrc”
```

2º. Borrar la carpeta “build” en caso en el que la tenga:

```
“rm -rf build”
```

3º. Crear una nueva “build” y movernos a ella:

```
“mkdir build && cd build”
```

4º. Compilar y generar ejecutables:

```
“cmake .. -DPICO_SDK_PATH=/home/sandra/.pico-sdk/sdk/2.1.0 -DPICO_BOARD=pico  
make -j4”
```

Una vez hemos completado esto, simplemente arrastraremos el archivo .uf2 a nuestra raspberry (“cp examples/deteccionCaracol/deteccionCaracol.uf2 /media/sandra/RPI-RP2/”) y observaremos si funciona a través de su consola (“screen /dev/ttyACM0 115200”).

Como podemos observar, por consola va a devolver un valor entero entre -128 y 127, cuanto más se aproxime al máximo quiere decir que mayor certeza tiene de que existe un caracol, en el caso contrario significa que no hay uno de estos.

Ahora que ya sabemos que funciona, vamos a probar a reducir el espacio reservado para el tensor arena. Sin embargo, observamos que el tamaño que le podemos dejar cómo mínimo es de 204.596 bytes. Esto quiere decir que el modelo está ocupando toda la RAM de la Pico (264KB), lo cuál hace que de momento funcione sin romper nada pero en el momento en el que añadamos la cámara esto no va a funcionar.

Para solucionar este problema, lo más conveniente es entrenar a la IA con imágenes en blanco y negro o reducir la resolución de las imágenes, sin embargo esto no es posible puesto que MobileNetV2 no puede ser modificado libremente, ha de ser modificado como se permite, y ya se encuentra reducido al máximo.

La solución más realista sería adaptar un modelo propio para que funcione lo mejor posible y repetir los pasos anteriores, esperando así solucionar los problemas de memoria.

Tras realizar esto, quedaría probar la cámara y ver si funciona todo en la vida real.